



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

INFORME VIDEOJUEGO SNAKE - UNIDAD II

Curso: Programación II

Docente: Breno Solim Luque Zúñiga

Ramirez Ajalla, Angie Belen (2024016759)

**Tacna – Perú
2026**

INFORME VIDEOJUEGO SNAKE

I. INFORMACIÓN GENERAL

Objetivos

- Aplicar programación orientada a objetos en C# para desarrollar un videojuego interactivo
- Utilizar Windows Forms para crear una interfaz gráfica funcional
- Implementar eventos y temporizadores para controlar el movimiento de la serpiente
- Desarrollar un sistema de puntaje y detección de colisiones
- Utilizar colecciones como List para almacenar el cuerpo de la serpiente

Equipos materiales programas y recursos utilizados

- Visual Studio
- Lenguaje de programación C#
- Windows Forms (.NET)
- Programación Orientada a Objetos

II. MARCO TEÓRICO

Programación Orientada a Objetos

La programación orientada a objetos es un paradigma de programación basado en el uso de clases y objetos. Permite organizar el código de manera más estructurada mediante atributos métodos encapsulamiento constructores y reutilización de código.

Windows Forms

Windows Forms es una biblioteca de .NET utilizada para desarrollar aplicaciones de escritorio con interfaces gráficas. Permite trabajar con botones paneles etiquetas temporizadores y eventos para crear programas interactivos.

Colecciones List

La colección List permite almacenar múltiples elementos dinámicamente. En el videojuego Snake se utilizó para guardar cada parte del cuerpo de la serpiente y controlar su crecimiento durante la partida.

Timer

El Timer es un componente que ejecuta instrucciones repetidamente en intervalos de tiempo definidos. En este proyecto se utilizó para actualizar continuamente el movimiento de la serpiente y el estado del juego.

III. PROCEDIMIENTO

Código:

Se importan las librerías necesarias para trabajar con formularios gráficos dibujo colecciones y eventos en Windows Forms. También se declaran las variables principales utilizadas durante la ejecución del videojuego como la serpiente la comida el puntaje el tamaño de los bloques y el control de inicio del juego.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Drawing;
4  using System.Windows.Forms;
5
6  namespace Juego_Snake
7  {
8      4 referencias
8      public partial class Form1 : Form
9      {
10         Snake snake;
11         Food food;
12
13         Random random = new Random();
14
15         int score = 0;
16         int size = 20;
17
18         bool gameStarted = false;
19     }
```

Se configura el formulario principal del videojuego Snake incluyendo la detección de teclado y la mejora gráfica. También se inicializa el juego creando la serpiente la comida el puntaje y el temporizador

```
1 referencia
20 public Form1()
21 {
22     InitializeComponent();
23
24     this.DoubleBuffered = true;
25     this.KeyPreview = true;
26
27     StartGame();
28 }
29
30 1 referencia
31 private void Form1_Load(object sender, EventArgs e)
32 {
33 }
34
35 3 referencias
36 private void StartGame()
37 {
38     snake = new Snake();
39
40     food = new Food(
41         random.Next(0, 39),
42         random.Next(0, 25));
43
44     score = 0;
45
46     lblPuntaje.Text = "Puntaje: 0";
47
48     gameStarted = false;
49
50     timer1.Interval = 100;
51
52     timer1.Stop();
53
54     panelJuego.Invalidate();
55 }
```

Se controla el movimiento automático de la serpiente mediante el temporizador. También se detecta cuando la serpiente consume comida aumenta el puntaje genera nueva comida y verifica colisiones contra los bordes o contra su propio cuerpo para finalizar la partida.

```
1 referencia
56 private void timer1_Tick(object sender, EventArgs e)
57 {
58     snake.Move();
59
60     // Comer comida
61     if (snake.Head.X == food.X &&
62         snake.Head.Y == food.Y)
63     {
64         snake.Grow();
65
66         score += 10;
67
68         lblPuntaje.Text = "Puntaje: " + score;
69
70         food = new Food(
71             random.Next(0, 39),
72             random.Next(0, 25));
73     }
74
75     // Colisión con bordes
76     if (snake.Head.X < 0 ||
77         snake.Head.Y < 0 ||
78         snake.Head.X >= 39 ||
79         snake.Head.Y >= 25)
80     {
81         GameOver();
82     }
83
84     // Colisión consigo misma
85     for (int i = 1; i < snake.Body.Count; i++)
86     {
87         if (snake.Head == snake.Body[i])
88         {
89             GameOver();
90         }
91     }
92
93     panelJuego.Invalidate();
94 }
```

En esta sección se controla el final de la partida mostrando un mensaje de Game Over y permitiendo reiniciar o cerrar el videojuego. También se incluyen los botones principales del sistema y el inicio del dibujo gráfico del tablero dentro del panel del juego

```
2 referencias
96 private void GameOver()
97 {
98     timer1.Stop();
99
100     DialogResult result = MessageBox.Show(
101         "GAME OVER\n¿Deseas reiniciar?",
102         "Snake",
103         MessageBoxButtons.YesNo);
104
105     if (result == DialogResult.Yes)
106     {
107         StartGame();
108     }
109     else
110     {
111         Application.Exit();
112     }
113 }
114
115 1 referencia
116 private void btnReiniciar_Click(object sender, EventArgs e)
117 {
118     StartGame();
119 }
120
121 1 referencia
122 private void btnSalir_Click(object sender, EventArgs e)
123 {
124     Application.Exit();
125 }
126
127 1 referencia
128 private void panelJuego_Paint(object sender, PaintEventArgs e)
129 {
130     Graphics g = e.Graphics;
```

En esta parte se dibuja la cuadrícula del tablero utilizando líneas verticales y horizontales. También se representa gráficamente la serpiente mediante bloques verdes y la comida utilizando una figura circular roja dentro del panel principal del videojuego

```
129 // Cuadrícula
130 Pen gridPen = new Pen(Color.FromArgb(40, 40, 40));
131
132 for (int i = 0; i < 800; i += 20)
133 {
134     g.DrawLine(gridPen, i, 0, i, 520);
135 }
136
137 for (int j = 0; j < 520; j += 20)
138 {
139     g.DrawLine(gridPen, 0, j, 800, j);
140 }
141
142 // Dibujar serpiente
143 foreach (Point p in snake.Body)
144 {
145     g.FillRectangle(
146         Brushes.LimeGreen,
147         p.X * size,
148         p.Y * size,
149         size,
150         size);
151 }
152
153 // Dibujar comida
154 g.FillEllipse(
155     Brushes.Red,
156     food.X * size,
157     food.Y * size,
158     size,
159     size);
160 }
161
```

En esta sección se controla el movimiento de la serpiente mediante las teclas W A S D. También se inicia automáticamente la partida al presionar una tecla por primera vez y se evita que la serpiente pueda moverse en dirección contraria a su recorrido actual.

```
162 private void Form1_KeyDown(object sender, KeyEventArgs e)
163 {
164     // Iniciar juego al presionar tecla
165     if (!gameStarted)
166     {
167         timer1.Start();
168         gameStarted = true;
169     }
170
171     switch (e.KeyCode)
172     {
173         case Keys.W:
174             if (snake.Direction != Direction.Down)
175             {
176                 snake.Direction = Direction.Up;
177             }
178
179             break;
180
181         case Keys.S:
182             if (snake.Direction != Direction.Up)
183             {
184                 snake.Direction = Direction.Down;
185             }
186
187             break;
188
189         case Keys.A:
190             if (snake.Direction != Direction.Right)
191             {
192                 snake.Direction = Direction.Left;
193             }
194
195             break;
196
197         case Keys.D:
198             if (snake.Direction != Direction.Left)
199             {
200                 snake.Direction = Direction.Right;
201             }
202
203             break;
204     }
205 }
206
207
208
209
210
211
```

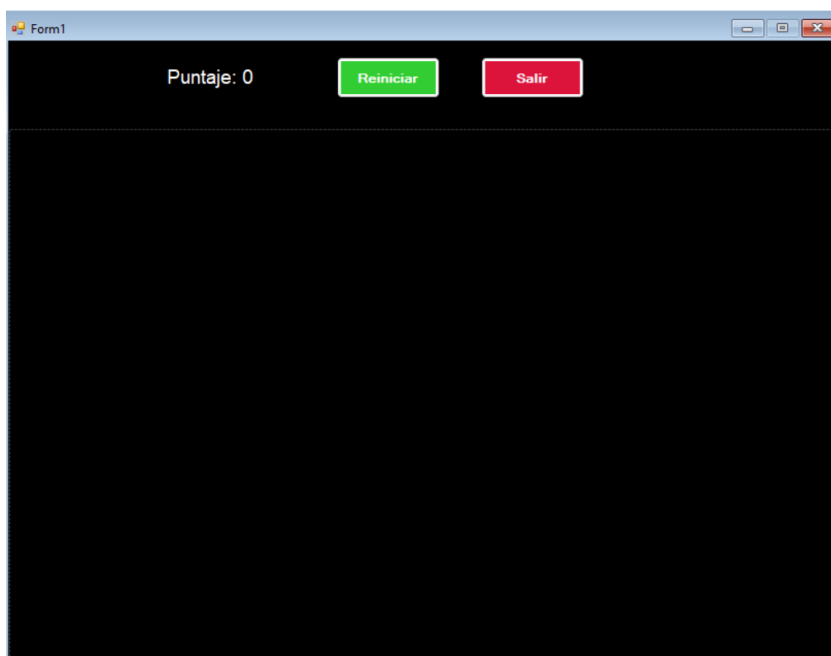

Se definen las direcciones disponibles para el movimiento de la serpiente mediante una enumeración. También se implementa una interfaz de movimiento y la clase principal de la serpiente donde se controla el cuerpo la dirección el crecimiento y el desplazamiento continuo dentro del tablero del videojuego.

```
212 public enum Direction
213 {
214     Up,
215     Down,
216     Left,
217     Right
218 }
219
220 1 referencia
220 public interface IMovable
221 {
222     2 referencias
222     void Move();
223 }
224 3 referencias
224 public class Snake : IMovable
225 {
226     9 referencias
226     public List<Point> Body { get; set; }
227     10 referencias
227     public Direction Direction { get; set; }
228     private bool grow = false;
229
230     8 referencias
230     public Point Head
231     {
232         get { return Body[0]; }
233     }
234     1 referencia
234     public Snake()
235     {
236         Body = new List<Point>();
237         Body.Add(new Point(10, 10));
238         Direction = Direction.Right;
239     }
240     2 referencias
240     public void Move()
241     {
242         Point newHead = Head;
243         switch (Direction)
244         {
245             case Direction.Up:
246                 newHead.Y--;
247                 break;
248             case Direction.Down:
249                 newHead.Y++;
250                 break;
251             case Direction.Left:
252                 newHead.X--;
253                 break;
254             case Direction.Right:
255                 newHead.X++;
256                 break;
257         }
258         Body.Insert(0, newHead);
259
260         if (!grow)
261         {
262             Body.RemoveAt(Body.Count - 1);
263         }
264         else
265         {
266             grow = false;
267         }
268     }
269 }
270
```

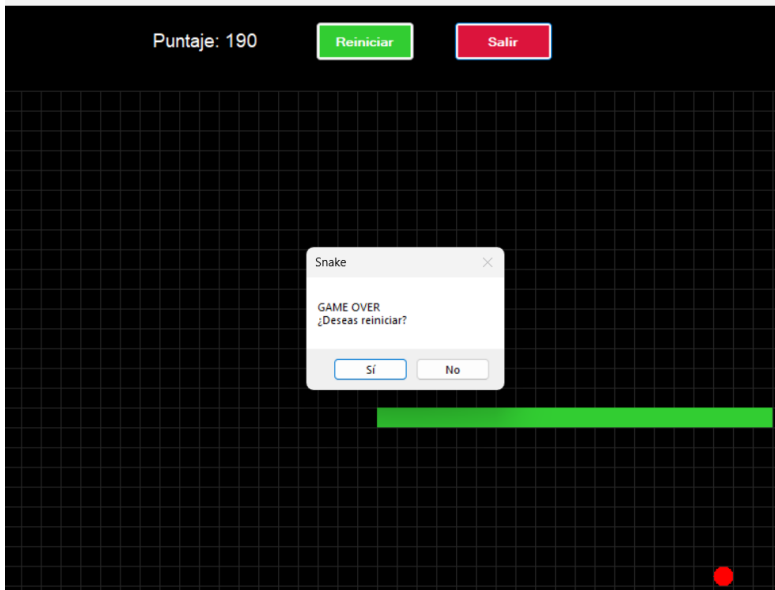
En esta última parte se implementa el crecimiento de la serpiente después de consumir la comida. También se define la clase Food encargada de almacenar las posiciones X y Y de la comida dentro del tablero mediante un constructor y atributos públicos.

```
271     1 referencia
272     public void Grow()
273     {
274         grow = true;
275     }
276
277     4 referencias
278     public class Food
279     {
280         3 referencias
281         public int X { get; set; }
282
283         3 referencias
284         public int Y { get; set; }
285
286         2 referencias
287         public Food(int x, int y)
288         {
289             X = x;
290             Y = y;
291         }
292     }
293 }
```

Ventana:



Ejecución:



IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

¿Qué indican los resultados?

- Los resultados muestran que el uso de programación orientada a objetos y Windows Forms facilita el desarrollo de videojuegos interactivos en C#.
- La aplicación permite controlar correctamente el movimiento de la serpiente detectar colisiones y administrar el puntaje del jugador de manera dinámica.

¿Qué se ha encontrado?

- Se comprobó que las colecciones List permiten almacenar el cuerpo de la serpiente eficientemente.
- Se observó que el Timer facilita la actualización continua del videojuego.
- Se evidenció que los eventos de teclado permiten una interacción dinámica con el usuario.
- Se comprobó que Windows Forms facilita el desarrollo de interfaces gráficas para videojuegos sencillos.

CONCLUSIONES

- Verificar que los nombres de los controles coincidan con los utilizados en el código
- Configurar correctamente los eventos del teclado y del temporizador para evitar errores de funcionamiento
- Validar correctamente las colisiones para evitar fallos durante la ejecución del videojuego